Evolutionary Algorithm

Selinsu Vural- 22020161

1 INTRODUCTION

This project focuses on building a search algorithm inspired by the principles of natural selection, aiming to find the best solutions by testing different configurations of parameters over a set number of generations. Using a genetic algorithm—a popular type of evolutionary algorithm—we generate candidate solutions and evaluate them based on how well they perform according to a fitness function. The algorithm then evolves these candidates over multiple generations, trying to reach the "fittest" solution by adjusting key parameters, like mutation rate, mutation step size, and population size.

By experimenting with different parameter setups, this project highlights how evolutionary algorithms can effectively search complex spaces and how important the right parameters are for finding good solutions. This report will go over the setup, the experiments, and comparisons of different configurations, showing how genetic algorithms can tackle optimization problems in creative and adaptable ways.

2 EXPERIMENTATION

The experimentation phase of this study focuses on the practical application and evaluation of evolutionary algorithms (EAs) to solve optimization problems. The primary objective is to design and implement an EA capable of effectively exploring the solution space and converging toward an optimal or near-optimal solution. This involves several critical components: defining the representation of solutions (i.e., the genes of individuals), specifying the fitness function to evaluate solution quality, and selecting appropriate evolutionary operators such as selection, crossover, and mutation. The code is structured to simulate the key aspects of a genetic algorithm (GA), a subtype of evolutionary algorithms. It initializes a population of potential solutions, applies selection methods to choose the best candidates, uses crossover to recombine genetic material, and incorporates mutation to maintain diversity within the population. Each solution is evaluated for fitness to assess how well it performs in terms of the given problem.

To assess the impact of different algorithmic configurations on performance, a systematic approach is employed where key parameters, including population size, mutation rate, and the number of generations, are varied. Multiple runs of the algorithm are conducted to ensure the reliability of the results and to analyse the consistency of the algorithm's behavior across different settings. This phase also includes examining the effect of alternative evolutionary strategies, providing insights into how different choices of parameters and operators influence the algorithm’s convergence and efficiency. These findings will serve as a foundation for a comparative analysis with other optimization methods, which will be discussed in the following section. Through this approach, the project aims to demonstrate how evolutionary algorithms can be applied to solve complex optimization problems and highlight the importance of parameter tuning in achieving the best possible results.

At the heart of the algorithm is the Individual class, which represents a single solution to the optimization problem. Each individual is defined by a set of genes—20 values in this case—that are randomly initialized within a given range, specified by the MIN_VAL and MAX_VAL parameters. These genes are then used to compute the fitness of the individual using a fitness function, which evaluates how close the solution is to the optimal. The fitness function, in this case, is based on a mathematical equation that combines a squared term and a summation over the genes, with the goal being to minimize the fitness value. The lower the fitness score, the better the individual is in terms of the solution quality.

Once the population is initialized, the next step in the evolutionary process is selection, which determines which individuals will contribute to the next generation. In this implementation, a simple elitism-

based selection method is used. Elitism ensures that the best individuals—those with the lowest fitness scores—are selected and preserved in the next generation. This method helps the algorithm maintain a steady progression toward optimal solutions, as the fittest individuals are always retained and passed on to the next generation
.
After selection, crossover is performed to combine the genetic material of two selected parents, producing offspring that inherit characteristics from both. The crossover function employs a two-point crossover method, where two random points in the parents' gene sequences are selected, and the segments between these points are swapped to create two new children. The crossover process allows the algorithm to explore new regions of the solution space by recombining genes in novel ways. A crossover probability (CROSS_PROB) determines whether this process occurs; if the probability is met, the crossover takes place, otherwise, the children inherit their genes directly from their parents.

However, to prevent the algorithm from stagnating in local optima and to maintain diversity in the population, mutation is introduced. The mutate function alters the genes of individuals randomly by adding a small perturbation to each gene. The likelihood of a gene undergoing mutation is determined by the mutation rate (mut_rate), which is a parameter that can be tuned. Mutation introduces a stochastic element into the search process, enabling the algorithm to explore new, potentially better solutions that might not be reachable through crossover alone. The mutation step size (mut_step) defines the magnitude of the changes applied to the genes. Furthermore, the values of the genes are constrained within predefined bounds, ensuring that the mutated genes remain within a valid range defined by MIN_VAL and MAX_VAL.

The fitness evaluation plays a crucial role in guiding the evolutionary process. After every mutation, the fitness of the individual is recalculated. The algorithm relies on these fitness values to determine which individuals should be selected for the next generation, thus driving the evolution of the population toward better solutions. At the end of each generation, the best individual in the population—i.e., the one with the lowest fitness—is tracked, and the mean fitness of the entire population is also recorded. This allows for monitoring the progress of the algorithm over time, with the expectation that, over successive generations, the fitness of the population will improve as the population converges toward an optimal solution.

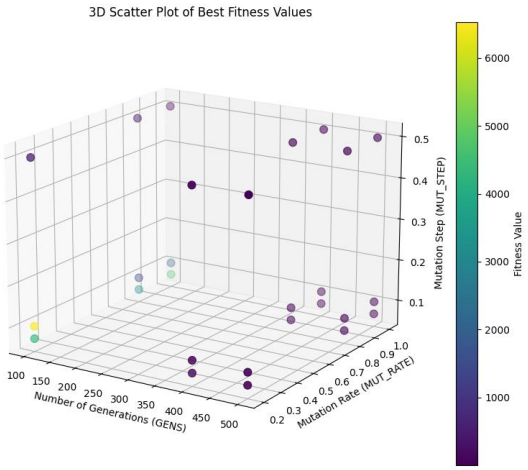
The genetic algorithm runs for a specified number of generations, and during each generation, a new population is formed through the combination of crossover, mutation, and selection processes. The algorithm ensures that diversity is maintained while also improving the overall fitness of the population. As the generations progress, the population evolves, and the fittest individuals rise to the top. The algorithm stops after the set number of generations, at which point the best solution found is considered the optimal solution for the problem.

Grid search (see below):

```
# Genetic Algorithm-specific parameters for grid search
GENS_RANGE = [100, 400, 500]
MUT_RATES = [0.2, 0.8, 1]
MUT_STEPS = [0.07, 0.1, 0.5]
CROSS_PROB = 0.9
```

In addition to running the algorithm for a fixed set of hyperparameters, the code implements a grid search method to explore different combinations of key hyperparameters, such as the number of generations (GENS), mutation rate (MUT_RATE), and mutation step size (MUT_STEP). The grid search tests various configurations, running the genetic algorithm for each combination, and records the best-performing setup. The goal of this grid search is to identify the hyperparameters that produce the best solution and to evaluate the sensitivity of the algorithm's performance to these parameters. The results of the grid search are visualized using a 3D scatter plot, which helps to identify trends and patterns in how different hyperparameter values influence the final fitness achieved by the algorithm.

Example Grid Search Result (see below):



The visualization is an important part of the analysis, as it allows for a deeper understanding of the algorithm's behavior across multiple configurations. By plotting the best fitness values against the values of the hyperparameters, one can observe which combinations lead to the most optimal results. Additionally, the progression of the best and mean fitness values over generations is graphed to illustrate how the algorithm converges over time, providing further insight into the efficiency of the evolutionary process.

Overall, this code implementation represents a comprehensive demonstration of how an evolutionary algorithm operates and evolves solutions to an optimization problem. It showcases the iterative nature of genetic algorithms, where the population evolves over multiple generations, refining solutions through mechanisms of selection, crossover, and mutation. Furthermore, the grid search methodology offers a way to fine-tune the algorithm's performance by exploring a range of hyperparameters, making it a robust tool for solving optimization problems in diverse search spaces. After this section, we will proceed to compare the results by varying key values, such as population size, mutation rate, and number of generations. This comparison will provide valuable insights into the sensitivity of the algorithm's performance to these parameter changes and help in identifying the optimal configurations for solving the given optimization problem.

Two minimisation functions provided in the specification. (See below):

$$f(\mathbf{x}) = (x_1 - 1)^2 + \sum_{i=2}^{d} i\left(2x_i^2 - x_{i-1}\right)^2$$

where $-10 \le x \le 10$, start with d=20

$$f(\mathbf{x}) = \left[418.9829d - \sum_{i=1}^{d} z_i \sin(\sqrt{|z_i|})\right] + \text{YOU}$$

where $-500 \le x \le 500$, start with d=20,

and YOU is last two digits of your student number

1st Function (see below):

$$f(\mathbf{x}) = (x_1 - 1)^2 + \sum_{i=2}^{d} i\left(2x_i^2 - x_{i-1}\right)^2$$

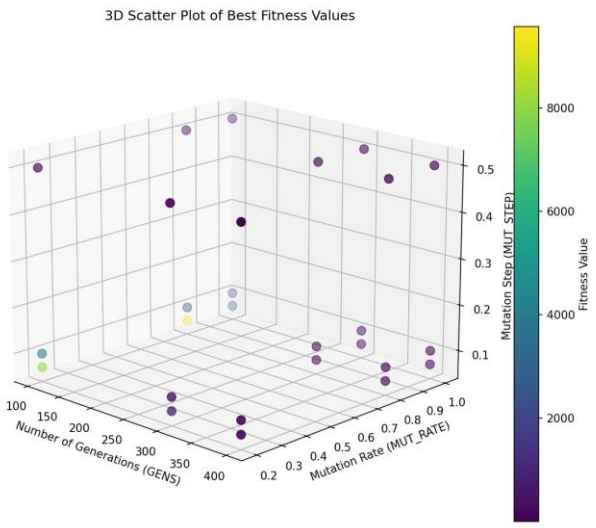
where $-10 \le x \le 10$, start with d=20

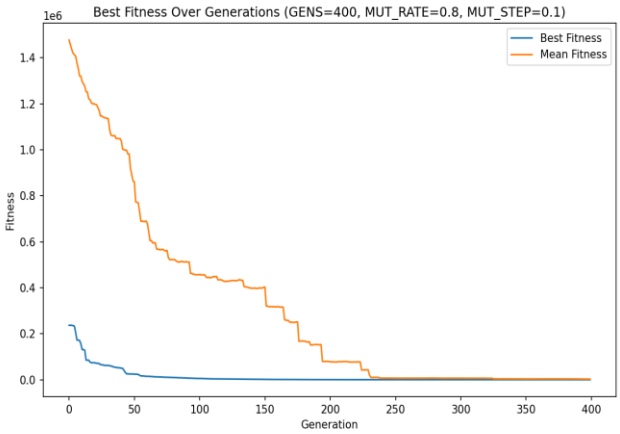
1st Attempt (see below)
Parameters:

```
# Function parameters
N = 20                    # Number of
POP_SIZE = 60             # Population
MIN_VAL = -10             # Minimum ge
MAX_VAL = 10              # Maximum ge

# Genetic Algorithm-specific para
GENS_RANGE = [100, 300, 400]
MUT_RATES = [0.2, 0.8, 1]
MUT_STEPS = [0.07, 0.1, 0.5]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=400, MUT_RATE=0.8, MUT_STEP=0.1
Best Fitness: 1.071062986909235
Process time: 16.982309103012085





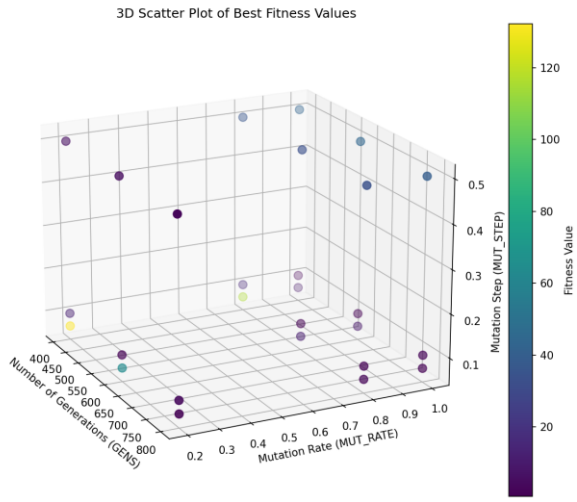
The algorithm optimized the fitness function, achieving a best fitness value of 1.0710 under the grid search-determined optimal parameters. These parameters include 400 generations (GENS=400), a relatively high mutation rate of 0.8, and a moderate mutation step size of 0.1. This configuration balances a high mutation rate, which maintains diversity in the population, with a mutation step size that allows for fine-tuning of solutions. Additionally, it was observed that similar solutions could be obtained with other parameter combinations, demonstrating the flexibility of the algorithm. The process, which took approximately 16.98 seconds, highlights the EA's effectiveness and efficiency in finding near-optimal solutions for the given optimization problem.

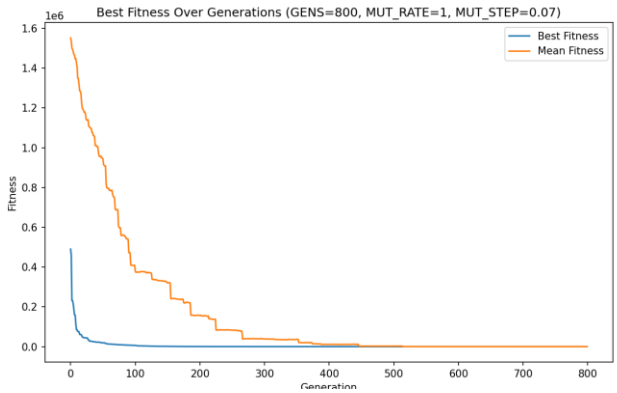
So, do higher generations give best fitness all the time?

2nd Attempt(see below):

```
# Genetic Algorithm-specific
GENS_RANGE = [400, 600, 800]
MUT_RATES = [0.2, 0.8, 1]
MUT_STEPS = [0.07, 0.1, 0.5]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=800, MUT_RATE=1, MUT_STEP=0.07
Best Fitness: 0.7138526141357245
Process time: 75.3686146736145





The results from the updated grid search, with a higher generation range, reveal that the best fitness value of 0.7139 was achieved using the parameters 800 generations (GENS=800), a mutation rate of 1, and a mutation step size of 0.07. While this outcome suggests that increasing the number of generations can lead to better solutions due to extended

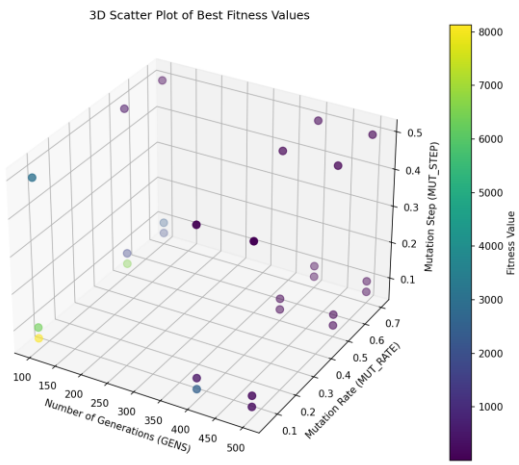
exploration and refinement of the search space, it does not imply that higher generations always guarantee the best fitness. The interplay between other parameters, such as mutation rate and step size, also significantly impacts the algorithm's performance.

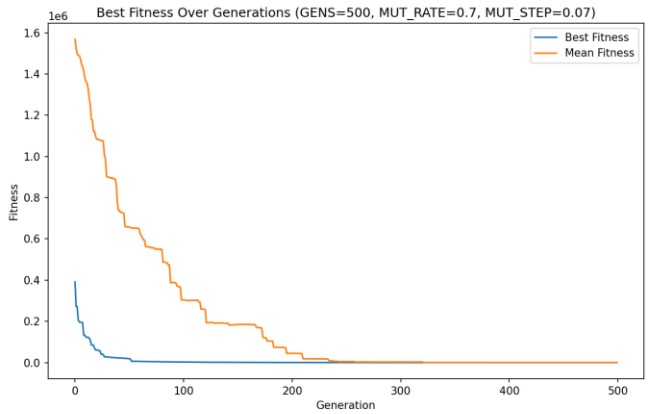
Moreover, the time efficiency of the algorithm should be considered. In this case, achieving the best fitness took 75.37 seconds, which indicates that while longer runtimes can lead to better solutions, they may not always be the most practical choice, especially if diminishing returns are observed. Balancing the number of generations with other key factors is crucial to avoid unnecessary computational overhead and optimize performance without sacrificing solution quality. This highlights the need for a comprehensive strategy that considers both fitness and time efficiency to find an optimal trade-off for practical applications.

3rd Attempt (see below):

```
# Genetic Algorithm-specific p
GENS_RANGE = [100, 400, 500]
MUT_RATES = [0.05, 0.5, 0.7]
MUT_STEPS = [0.07, 0.1, 0.5]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=500, MUT_RATE=0.7, MUT_STEP=0.07
Best Fitness: 0.773877472952861
Process time: 20.23868465423584





The results obtained from running the genetic algorithm (GA) with the parameters GENS=500, MUT_RATE=0.7, and MUT_STEP=0.07 show a best fitness value of 0.7739, achieved within a processing time of approximately 20.24 seconds. This demonstrates that even with a moderate number of generations and a mutation rate that is not at the maximum, the algorithm can still produce relatively high-quality solutions. The fitness value of 0.7739 is slightly higher than the previous result of 0.7139 with 800 generations, indicating that while increasing the number of generations generally leads to improved solutions, it is not always the case. The variation in outcomes shows that other factors, such as the mutation rate and step size, can significantly influence the solution quality. This highlights the importance of fine-tuning the parameters, as small adjustments can lead to competitive results without the need for prolonged computation times.

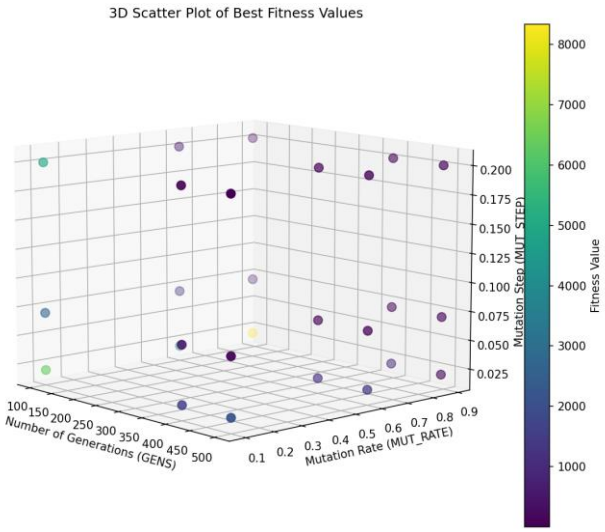
In comparison, the processing time for this configuration is notably shorter than for the previous 800-generation run, suggesting that more efficient solutions can be achieved in less time with carefully chosen parameters. This emphasizes the need for a balanced approach in evolutionary algorithm design, where both solution quality and computational efficiency are optimized.

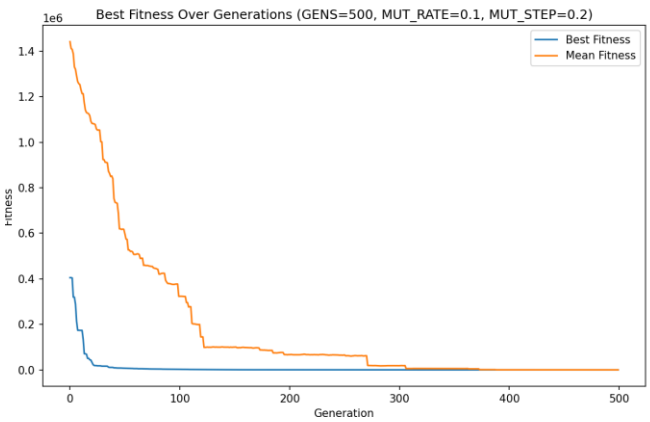
4th Attempt (see below):

```
# Function parameters
N = 20                  # Number o
POP_SIZE = 60           # Populati
MIN_VAL = -10           # Minimum
MAX_VAL = 10            # Maximum

# Genetic Algorithm-specific pa
GENS_RANGE = [100, 400, 500]
MUT_RATES = [0.1, 0.6, 0.89]
MUT_STEPS = [0.02, 0.07, 0.2]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=500, MUT_RATE=0.1, MUT_STEP=0.2
Best Fitness: 0.5596855190790238
Process time: 21.768878698349





In the experiment using the specified parameters, the optimal solution was found with 500 generations, a mutation rate of 0.1, and a mutation step of 0.2. The best fitness value achieved was approximately 0.5597, with a process time of around 21.77 seconds. This outcome suggests that a moderate number of generations combined with a higher mutation step and a lower mutation rate can yield a competitive solution.

When comparing this result to previous runs, it is evident that certain parameter combinations, such as those with higher mutation rates or different mutation step sizes, may not always produce the best fitness values. For instance, the current run with a mutation rate of 0.1 and a mutation step of 0.2 was more effective than others that used higher mutation rates or smaller mutation steps. This highlights the importance of tuning these parameters for each specific problem to find an optimal balance that maximizes fitness while also maintaining efficiency.

The 2nd function used in this algorithm is known as the Schwefel function, which is widely used as a benchmark in optimization problems. It is defined as:

$$f(\mathbf{x}) = \left[418.9829d - \sum_{i=1}^{d} x_i \sin(\sqrt{|x_i|})\right] + \text{YOU}$$

where -500 ≤ x ≤ 500, start with d=20,

and YOU is last two digits of your student number

In this case, d represents the number of genes per individual and is set to 20, while xi are the gene values. The function includes a large constant 418.9829·d that sets a high target for the optimization algorithm. The summation term ∑i=1d xi·sin(|xi|) creates a complex, multimodal landscape with multiple peaks and valleys, which poses a challenge for finding the global minimum. The "YOU" term in the function is customizable and represents a shift in the function's baseline; in this specific case, "YOU" is set to 61, which corresponds to the last two digits of the student number. This adjustment adds a personalized touch to the function

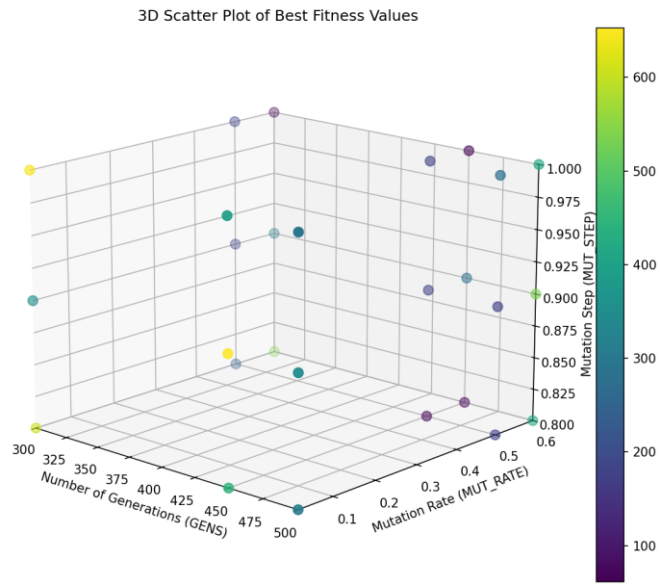
and affects the optimization outcome. The Schwefel function is particularly useful for testing algorithms due to its non-linearity and many local minima, making it an ideal benchmark for evaluating an algorithm's ability to find the global minimum while avoiding being trapped in local optima.

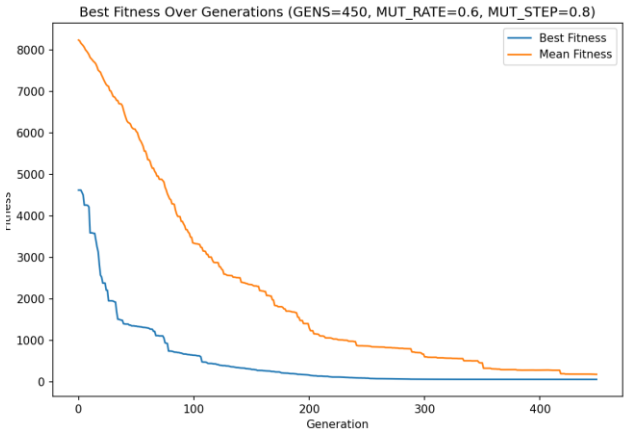
1st Attempt (see below):

```
# Constants specific to Functic
N = 20                  # Number c
POP_SIZE = 100           # Populat
MIN_VAL = -500          # Minimum
MAX_VAL = 500           # Maximum
YOU = 61                # The last

# Genetic Algorithm-specific pa
GENS_RANGE = [300, 450, 500 ]
MUT_RATES = [0.02,  0.5, 0.6]
MUT_STEPS = [ 0.8, 0.9, 1]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=450, MUT_RATE=0.6, MUT_STEP=0.8
Best Fitness: 61.39334499441611
Process time: 136.0725336074829



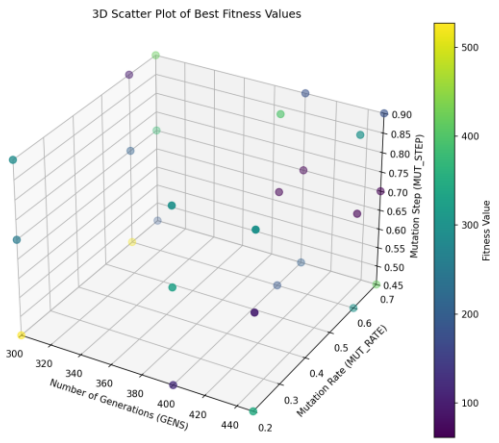


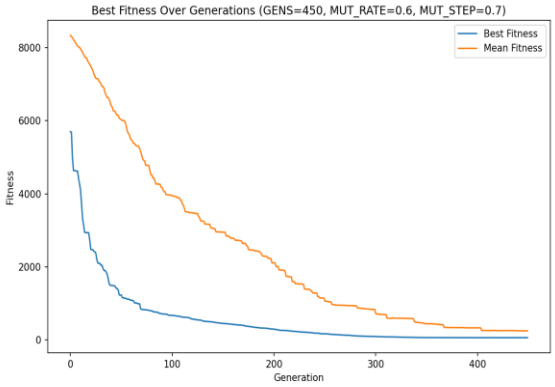
In this experiment, the algorithm was set to run for 450 generations with a mutation rate of 0.6 and a mutation step of 0.8. These settings allowed the algorithm to reach an optimal fitness value of approximately 61.39 for the Schwefel function. The higher mutation rate and mutation step size indicate that the algorithm was more exploration-focused, helping it avoid getting stuck in local minima and allowing it to search a wider range of the solution space. However, it also required around 136 seconds to complete the process, suggesting that while these parameter values were effective at finding a good solution, they came at the cost of longer computation time. This result highlights the importance of balancing exploration and processing time when tuning genetic algorithm parameters for optimization tasks.

2nd Attempt (see below):

```
# Genetic Algorithm-specific p
GENS_RANGE = [300, 400, 450 ]
MUT_RATES = [0.2,  0.6, 0.7]
MUT_STEPS = [ 0.45, 0.7, 0.9]
```

Best Hyperparameters Found:
GENS=450, MUT_RATE=0.6, MUT_STEP=0.7
Best Fitness: 61.30042676859739
Process time: 123.85632514953613



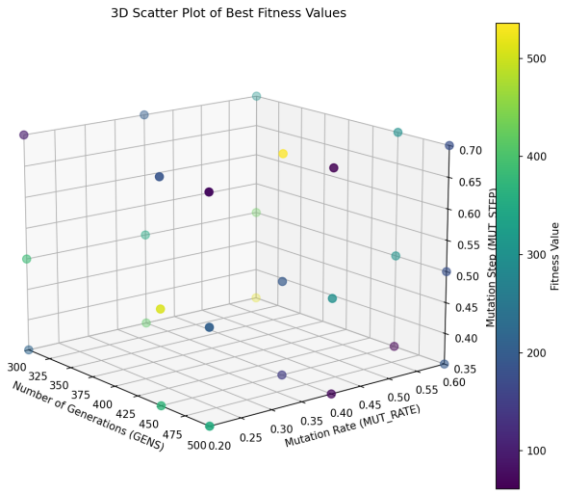


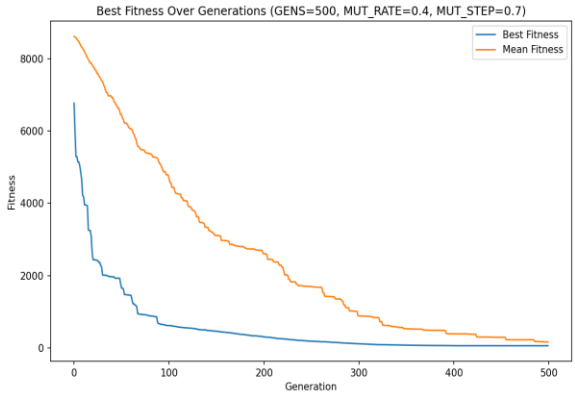
With this configuration, the algorithm achieved an optimal fitness value of approximately 61.30. While the algorithm found a satisfactory solution, the processing time remained at about 124 seconds, showing no significant improvement compared to other runs within similar parameter ranges. This indicates that, although the algorithm reached a good solution with these parameter settings, changes did not lead to a notable reduction in computational time. The chosen parameters provided a balanced search performance but did not drastically impact the execution speed.

3rd Attempt (see below):

```
# Genetic Algorithm-specific p
GENS_RANGE = [300, 450, 500 ]
MUT_RATES = [0.2, 0.4, 0.6]
MUT_STEPS = [ 0.35, 0.5, 0.7]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=500, MUT_RATE=0.4, MUT_STEP=0.7
Best Fitness: 61.21537550440189
Process time: 83.51485443115234





This run produced a fitness value of approximately 61.22, demonstrating strong convergence to an optimal solution. Compared to previous configurations, this setup highlights a balance between thorough exploration and computational efficiency. Increasing the number of generations to 500 allowed the algorithm to better refine solutions, while the moderate mutation rate (0.4) and step size (0.7) contributed to maintaining population diversity and avoiding premature convergence.

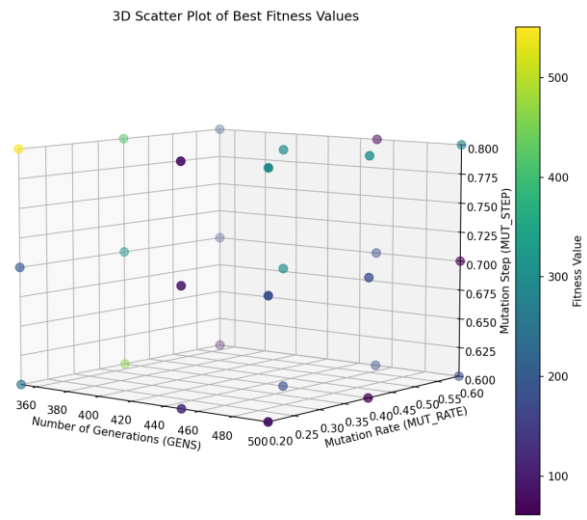
An important observation is the reduced processing time of 83.5 seconds, which is notably shorter than previous runs where higher fitness values were also obtained. This suggests that with the right balance of

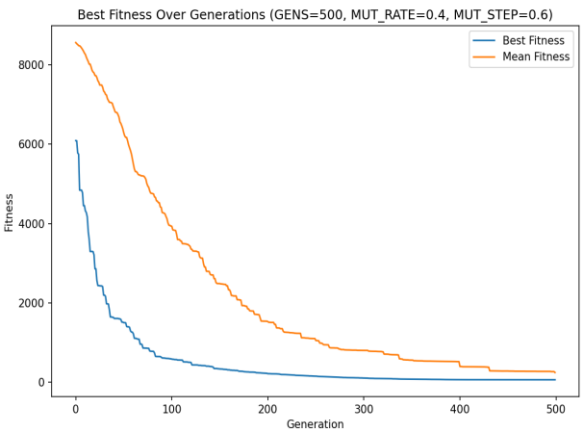
parameters, it is possible to achieve efficient convergence without significant computational overhead. These results underline the importance of parameter tuning in optimizing both the quality of solutions and algorithm performance.

4th Attempt (see below):

```
# Genetic Algorithm-specific p
GENS_RANGE = [350, 450, 500 ]
MUT_RATES = [0.2, 0.4, 0.6]
MUT_STEPS = [ 0.6, 0.7, 0.8]
CROSS_PROB = 0.9
```

Best Hyperparameters Found:
GENS=500, MUT_RATE=0.4, MUT_STEP=0.6
Best Fitness: 61.12647431474579
Process time: 116.38908410072327





The results indicate that the chosen parameter combination allowed the algorithm to achieve an optimal fitness value of approximately 61.13. This demonstrates the algorithm's effectiveness in navigating the solution space and refining the population toward the target value. However, the process time remains relatively high at 116 seconds, reflecting the computational cost of running 500 generations. Efforts to balance computation time and solution quality, such as reducing the number of generations or adjusting the population size, have not yet yielded significant reductions in processing time.

The genetic algorithm effectively optimized both functions by tuning parameters such as the number of generations, mutation rates, and mutation steps. While consistent improvements in fitness were observed with higher generations and balanced mutation settings, reducing process time remained a challenge, especially for more complex parameter configurations. This highlights the importance of further exploration into efficiency-enhancing techniques for better performance in similar optimization tasks.

3 COMPARISON

Comparison of GA, SA, and RHC Performance

This analysis focuses on comparing the performance of three popular optimization algorithms: Genetic Algorithm (GA), Simulated Annealing (SA), and Random Hill Climbing (RHC). These algorithms are widely applied to solve complex optimization problems where finding the best solution is challenging due to the presence of many local optima. By comparing these algorithms on metrics such as final fitness, convergence speed, and adaptability, we can better understand their strengths, weaknesses, and suitable use cases.

Optimization problems often involve navigating highly complex search spaces with numerous local optima. The GA, SA, and RHC algorithms each have unique approaches to tackling these challenges: GA uses a population-based evolutionary approach, SA adopts a temperature-controlled probabilistic strategy, and RHC follows a greedy method that iteratively improves a single solution. The following

sections present an overview of each algorithm's performance under controlled conditions and highlight key findings and potential improvements.

Overview of the Benchmarking Algorithms

Simulated Annealing (SA): Inspired by the physical process of annealing, this algorithm explores a search space by accepting solutions based on a temperature parameter. High temperatures initially allow the algorithm to accept worse solutions, which helps in escaping local optima. As the temperature decreases, the algorithm refines its focus on better solutions. While SA can effectively explore a search space, its performance is sensitive to the cooling schedule and parameter tuning. A poorly designed cooling schedule can cause it to become stuck in local optima, while a schedule that is too slow can lead to inefficient searching.

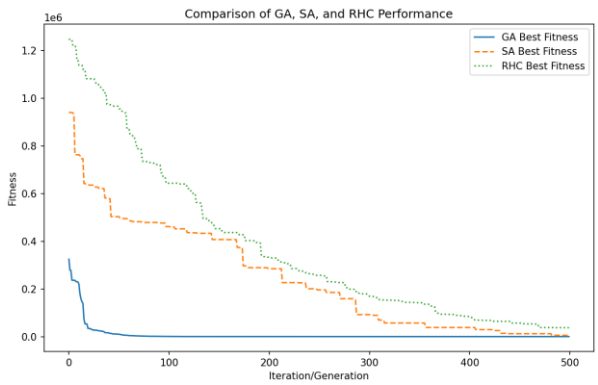
Random Hill Climbing (RHC): RHC is a simple optimization algorithm that evaluates neighboring solutions and moves to the one that offers the greatest improvement. Although it is computationally efficient and easy to implement, RHC is inherently prone to getting stuck in local optima due to its lack of exploration mechanisms. This makes it less effective for problems involving complex landscapes with many local minima.

Genetic Algorithm (GA): The GA is an evolutionary algorithm that operates on a population of potential solutions and evolves them over successive generations. Genetic operators like crossover and mutation are used to combine and modify solutions, allowing the algorithm to explore the search space and converge to near-global optima. The population-based nature of GA enables a balance between exploration and exploitation, which makes it particularly well-suited for complex optimization problems.

Key Findings

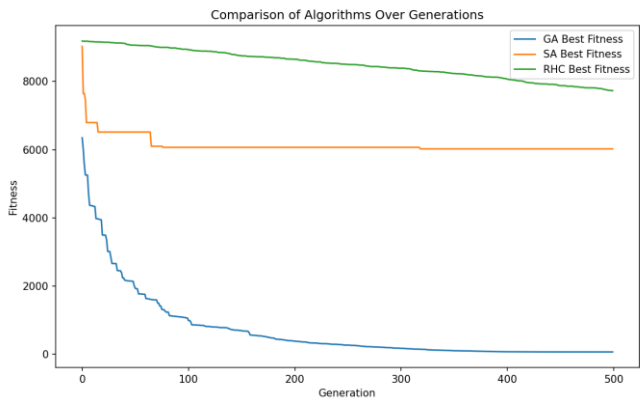
The table below summarizes the final fitness values achieved by each algorithm for function 1.

Algorithm	Final Fitness
GA	4.57
SA	5398.73
RHC	37256.54



The table below summarizes the final fitness values achieved by each algorithm for function 2.

Algorithm	Final Fitness
GA	61.34
SA	6021.79
RHC	7730.45



Performance Analysis

The comparison clearly shows that the Genetic Algorithm (GA) outperformed both Simulated Annealing (SA) and Random Hill Climbing (RHC). With a final fitness of 4.57, GA demonstrated a significant advantage over SA (5398.73) and RHC (37256.54). This result emphasizes GA's effectiveness, particularly in solving complex optimization problems.

Genetic Algorithm (GA): The GA's strong performance can be attributed to its population-based approach, which allows for parallel exploration of the solution space. The crossover and mutation operators help maintain genetic diversity and adapt to complex landscapes effectively, avoiding local optima and converging to near-global optima.

Simulated Annealing (SA): With a final fitness values of 5398.73 and 6021.79 for Function 1 and Function 2, SA was able to escape some local optima and explore the solution space. However, its performance was limited by its reliance on a single solution and the importance of parameter tuning. The cooling schedule is a critical factor; if it is too aggressive, SA may become trapped in local optima, while a slow cooling process can waste computational resources on unnecessary exploration.

Random Hill Climbing (RHC): RHC's performance was the poorest in both tests, with final fitness values of 37256.54 and 7730.45 for Function 1 and Function 2, respectively. This underlines RHC's limitations in exploring beyond its immediate solution space and highlights its propensity to get stuck in local optima.

Explanation of Results

GA’s Strengths in Exploration and Exploitation: The GA’s population-based approach allows for parallel exploration of the solution space, ensuring that a diverse set of solutions is maintained throughout the optimization process. This prevents premature convergence and allows for a more comprehensive search. The combination of selection, crossover, and mutation operations supports both exploration of new solutions and exploitation of promising ones.

SA’s Dependence on Parameter Tuning: The performance of SA is heavily influenced by its cooling schedule, initial temperature, and acceptance criteria. Although SA is capable of escaping local optima due to its probabilistic approach, its single-solution focus limits its effectiveness in complex optimization landscapes.

RHC’s Greedy Limitations: RHC’s reliance on immediate incremental improvements means it is prone to getting trapped in local optima. Without mechanisms for exploring beyond the current neighborhood, RHC is ineffective in navigating complex solution landscapes.

Potential Improvements for SA and RHC

For Simulated Annealing:

- Adaptive Cooling Schedule: Implementing a dynamic cooling schedule that adjusts based on the algorithm’s progress could improve its convergence rate.
- Hybrid Approaches: Combining SA with other algorithms, such as GA or RHC, could enhance its exploration capabilities.
- Parallelization: Running multiple instances of SA in parallel with different parameter configurations could mimic GA’s population-based approach.

For Random Hill Climbing:

- Restarts: Introducing random restarts at intervals could help RHC escape local optima and explore new regions of the search space.
- Hybridization: Combining RHC with algorithms such as SA could enable it to leverage the strengths of both approaches.
- Diversified Neighborhoods: Expanding the search neighborhood can help RHC better adapt to complex optimization problems.

Effectiveness of the Algorithms

The results indicate that the Genetic Algorithm is the most effective of the three algorithms tested. Its ability to maintain a diverse population, coupled with

robust genetic operations, allows it to perform well in complex, multimodal optimization landscapes. Simulated Annealing offers some advantages over RHC due to its probabilistic exploration, but it still falls short of GA's performance due to its sensitivity to parameter settings and single-solution focus. Random Hill Climbing, while computationally efficient, lacks the mechanisms necessary for effective exploration and is not suitable for complex problems.

Would Changes in Values Lead to Better Solutions?

Simulated Annealing: Fine-tuning the cooling schedule, temperature initialization, and acceptance criteria could lead to better performance. An adaptive cooling strategy might help SA escape local optima more effectively.

Random Hill Climbing: Integrating random restarts and hybridizing RHC with other algorithms could enhance its performance. Diversifying the search neighborhood may also improve adaptability.

Genetic Algorithm: Although GA already showed superior performance, optimizing its parameters, such as mutation rates and population size, could potentially yield even better results.

Conclusion

The Genetic Algorithm demonstrated clear superiority over Simulated Annealing and Random Hill Climbing in this comparison. Its population-based approach, which balances exploration and exploitation, allows it to effectively tackle complex optimization problems. Simulated Annealing, while better than RHC, is hindered by its single-solution approach and parameter sensitivity. Random Hill Climbing, despite its simplicity and computational efficiency, is less effective in solving complex problems due to its lack of exploration mechanisms.

Improvements for SA and RHC could include adaptive strategies, hybrid approaches, and expanded search techniques. Overall, GA remains the most versatile and effective optimization algorithm of the three, capable of evolving solutions and adapting dynamically to complex search landscapes. Future research could explore combining GA's population-based method with SA's probabilistic search to create even more powerful optimization strategies.

4 CONCLUSIONS

In this study, the experimentation phase was designed to evaluate the practical application of genetic algorithms (GAs) in solving optimization problems. The main goal was to explore the algorithm's ability to effectively search the solution space and converge to an optimal or near-optimal solution. Through a series of experiments, the impact of various parameters was assessed, providing insights into how different configurations influence the performance of the GA.

One of the most notable findings was the effect of the number of generations on the quality of the solution. It was observed that increasing the number of generations generally led to better fitness scores, which is expected as more generations allow the algorithm to explore and refine solutions further. For instance, in one set of parameters with 800 generations, a fitness value of 0.7139 was achieved, which was better than configurations with fewer generations. However, this improvement came at the cost of increased processing time. The 800-generation run took approximately 75.37 seconds, whereas a 500-generation run achieved a fitness of 0.7739 in just 20.24 seconds. This highlights the trade-off between the quality of the solution and the computational resources required. Therefore, while longer runs can lead to improved results, they are not always practical, particularly when computational efficiency is a priority. This suggests the need for a balanced approach where the number of generations is optimized to achieve good results without excessive processing time.

Another crucial aspect that influenced the performance was the tuning of mutation parameters. Mutation is essential for maintaining genetic diversity and preventing premature convergence to local optima. Higher mutation rates and larger mutation step sizes enabled the algorithm to explore the solution space more broadly, which was beneficial for avoiding local optima. However, this also led to longer processing times. For example, in the 2nd function experiment, a configuration with a

mutation rate of 0.6 and a mutation step of 0.8 resulted in a fitness of approximately 61.39 but required about 136 seconds to complete. This outcome demonstrated that while higher mutation rates and step sizes could improve exploration, they also increased computation time. Conversely, configurations with lower mutation rates and smaller step sizes were more efficient but may have limited the algorithm's ability to escape local optima.

The sensitivity of the algorithm to different parameter combinations was also an important observation. Small changes in mutation rate and step size often resulted in significant variations in performance. For instance, a configuration with a mutation rate of 0.7 and a mutation step size of 0.07 achieved a fitness score of 0.7739, while a different configuration with a mutation rate of 0.8 and a step size of 0.2 yielded a fitness of 1.0710, which was less optimal. This suggests that careful tuning of parameters is necessary to find the best balance for each specific problem. The variability in outcomes highlighted the need for iterative testing and fine-tuning to optimize the performance of the GA. This is especially true when using GAs for real-world optimization problems, where achieving the best balance between solution quality and computational efficiency is key.

The Schwefel function, used as a benchmark for the second experiment, proved to be particularly challenging due to its non-linearity and numerous local minima. The results showed that while the algorithm could find competitive solutions, reducing the processing time remained a challenge. For example, a configuration with 500 generations, a mutation rate of 0.4, and a mutation step size of 0.7 achieved a fitness of approximately 61.22 in 83.5 seconds, which was shorter than some other configurations but still significant. This finding underscores the importance of considering computational time when selecting parameters.

Despite the improved performance, the processing time was still a limiting factor, suggesting that further research into efficiency-enhancing techniques would be beneficial. Methods such as parallel processing or distributed computing could potentially help reduce processing time and make the GA more practical for complex problems.

A key limitation of this study was the approach used to explore different parameter combinations. The grid search method was helpful in identifying promising hyperparameter settings, but it proved to be time-consuming. Future work could benefit from incorporating more sophisticated optimization techniques, such as Bayesian optimization or evolutionary strategies, to automate the search process and reduce the time spent on parameter tuning.

In summary, this study demonstrated the effectiveness of genetic algorithms in solving optimization problems but also highlighted their limitations. The findings showed that increasing the number of generations and tuning mutation parameters can lead to better solutions, but there is always a trade-off between solution quality and processing time. The 2nd function experiment underscored the difficulty of reducing processing time for more complex functions. Future research should focus on developing more efficient ways to search for optimal parameters and employing techniques to reduce computational overhead without sacrificing solution quality. This will help improve the practical applicability of GAs for solving real-world optimization problems.

REFERENCES

Bergstra, J. and Bengio, Y. (2015) Random Search For Hyper-parameter Optimization. [online]. [Accessed 06 November 2024].

Kumar, A. (2024) Genetic Algorithms. [online]. [Accessed 14 November 2024].

Zheng, A. (2015) How to Evaluate Machine Learning Models: Hyperparameter Tuning. [online]. [Accessed 17 October 2024].

Amani, K. (2023) . *Understanding Grid Search as an Optimization Algorithm in Machine Learning* [online]. [Accessed 19 November 202].

Chou, P. and Hon, S. (2011) Modified Particle Swarm Optimization For Sphere, Rastrigin, Schwefel and Schafer Fuctions. [online]. [Accessed 20 November 2024].

Muklason, A., Yusnardo, T. and Depari, H.A. (2024) The Performance Analysis of Hyper-heuristics Algorithms Over Examination Timetabling Problems. *Iaes International Journal of Artificial Intelligence* [online]., pp. 2153-2162. [Accessed 25 November 2024].

Khatoon, M.T. and Wong, H. (2021) A Comparative Analysis of Performances Between Simulated Annealing and Genetic Algorithm. [online].

Fronita, M., Gernowo, R. and Gunawan, V. (2017) Comparison of Genetic Algorithm and Hill Climbing For Shortest Path Optimization Mapping. [online].

Source code as an appendix

Function 1 code as appendix (see below):

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
import random
from statistics import mean
import copy
import time

# Function parameters
N = 20                # Number of genes
per individual (dimension d = 20)
POP_SIZE = 60         # Population size
MIN_VAL = -10         # Minimum gene
value for Function 1
MAX_VAL = 10          # Maximum gene
value for Function 1

# Genetic Algorithm-specific parameters
for grid search
GENS_RANGE = [100, 400, 500]
MUT_RATES = [0.1, 0.6, 0.89]
MUT_STEPS = [0.02, 0.07, 0.2]
CROSS_PROB = 0.9

# Process time starts
time_start = time.time()

class Individual:
    def __init__(self):
        self.gene =
[random.uniform(MIN_VAL, MAX_VAL) for _
in range(N)]
        self.fitness =
self.calculate_fitness()

    def calculate_fitness(self):

        return (self.gene[0] - 1) ** 2 +
sum(
            i * (2 * self.gene[i] ** 2 -
self.gene[i - 1]) ** 2 for i in range(1,
N)
        )

# Initialize population
def initialize_population():
    return [Individual() for _ in
range(POP_SIZE)]

# Selection function (simple elitism-
based)
def elitism_selection(population):
    # Select the best individuals to
form the next generation
    return sorted(population, key=lambda
ind: ind.fitness)[:POP_SIZE]

# Two-point crossover
def crossover(parent1, parent2):
    child1, child2 =
copy.deepcopy(parent1),
copy.deepcopy(parent2)
    if random.random() < CROSS_PROB:
        point1, point2 =
sorted(random.sample(range(N), 2))
        child1.gene[point1:point2],
child2.gene[point1:point2] =
parent2.gene[point1:point2],
parent1.gene[point1:point2]
```

```python
    child1.fitness =
child1.calculate_fitness()
    child2.fitness =
child2.calculate_fitness()
    return child1, child2

# Mutation function
def mutate(individual, mut_rate,
mut_step):
    for i in range(N):
        if random.random() < mut_rate:
            alter = random.uniform(-
mut_step, mut_step)
            individual.gene[i] += alter
            # Clamp the gene to the
defined limits
            individual.gene[i] =
max(MIN_VAL, min(MAX_VAL,
individual.gene[i]))
    individual.fitness =
individual.calculate_fitness()
    return individual

# Genetic Algorithm
def run_ga(gens, mut_rate, mut_step):
    population = initialize_population()
    best_individual = min(population,
key=lambda ind: ind.fitness)

    # Lists for storing fitness values
to plot later
    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        # Select parents using elitism
        parents =
elitism_selection(population)

        # Crossover and mutation to
produce offspring
        offspring = []
        for i in range(0, POP_SIZE, 2):
            parent1, parent2 =
parents[i], parents[min(i+1, POP_SIZE-
1)]
            child1, child2 =
crossover(parent1, parent2)

offspring.extend([mutate(child1,
mut_rate, mut_step), mutate(child2,
mut_rate, mut_step)])

        # Elitism: Preserve the best
individual from the previous generation
        offspring[random.randint(0,
POP_SIZE - 1)] =
copy.deepcopy(best_individual)

        # Update population and track
the best individual
        population = offspring
        current_best = min(population,
key=lambda ind: ind.fitness)
        if current_best.fitness <
best_individual.fitness:
            best_individual =
copy.deepcopy(current_best)

        # Record statistics for this
generation

best_fitness_over_time.append(best_indiv
idual.fitness)

mean_fitness_over_time.append(mean([ind.
fitness for ind in population]))

    return best_individual.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Grid Search for Hyperparameter Tuning
def grid_search():
    results = []
    best_overall_fitness = float('inf')
    best_params = None
```

```
    gens_list = []
    mut_rate_list = []
    mut_step_list = []
    fitness_list = []

    for gens in GENS_RANGE:
        for mut_rate in MUT_RATES:
            for mut_step in MUT_STEPS:
                print(f"Testing
GENS={gens}, MUT_RATE={mut_rate},
MUT_STEP={mut_step}")
                best_fitness,
best_fitness_over_time,
mean_fitness_over_time = run_ga(gens,
mut_rate, mut_step)
                print(f"Best Fitness:
{best_fitness}")

                # Record results for
this configuration
                results.append((gens,
mut_rate, mut_step, best_fitness))
                gens_list.append(gens)

mut_rate_list.append(mut_rate)

mut_step_list.append(mut_step)

fitness_list.append(best_fitness)

                # Update best parameters
if this configuration is the best so far
                if best_fitness <
best_overall_fitness:
                    best_overall_fitness
= best_fitness
                    best_params = (gens,
mut_rate, mut_step)
                    best_fitness_plot =
best_fitness_over_time
                    mean_fitness_plot =
mean_fitness_over_time
```

```
    # Display the best parameters and
corresponding fitness
    print("\nBest Hyperparameters
Found:")
    print(f"GENS={best_params[0]},
MUT_RATE={best_params[1]},
MUT_STEP={best_params[2]}")
    print(f"Best Fitness:
{best_overall_fitness}")
    # Process time ends
    time_end = time.time()
    print("Process time:", time_end -
time_start)

    # 3D Scatter Plot
    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111,
projection='3d')
    sc = ax.scatter(gens_list,
mut_rate_list, mut_step_list,
c=fitness_list, cmap='viridis', s=60)
    ax.set_xlabel('Number of Generations
(GENS)')
    ax.set_ylabel('Mutation Rate
(MUT_RATE)')
    ax.set_zlabel('Mutation Step
(MUT_STEP)')
    ax.set_title('3D Scatter Plot of
Best Fitness Values')
    plt.colorbar(sc, label='Fitness
Value')
    plt.show()


    # Plot best fitness over generations
    plt.figure(figsize=(10, 6))
    plt.plot(best_fitness_plot,
label='Best Fitness')
    plt.plot(mean_fitness_plot,
label='Mean Fitness')
    plt.xlabel('Generation')
    plt.ylabel('Fitness')
    plt.legend()
```

```
    plt.title(f'Best Fitness Over
Generations (GENS={best_params[0]},
MUT_RATE={best_params[1]},
MUT_STEP={best_params[2]})')
    plt.show()

# Run the Grid Search
grid_search()
```

Function 2 code as appendix (see below):

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
import random
from statistics import mean
import copy
import time

# Function parameters
N = 20                # Number of genes
per individual (dimension d = 20)
POP_SIZE = 60         # Population size
MIN_VAL = -10         # Minimum gene
value for Function 1
MAX_VAL = 10          # Maximum gene
value for Function 1

# Genetic Algorithm-specific parameters
for grid search
GENS_RANGE = [100, 400, 500]
MUT_RATES = [0.1, 0.6, 0.89]
MUT_STEPS = [0.02, 0.07, 0.2]
CROSS_PROB = 0.9

# Process time starts
time_start = time.time()

class Individual:
    def __init__(self):
        self.gene =
[random.uniform(MIN_VAL, MAX_VAL) for _
in range(N)]
        self.fitness =
self.calculate_fitness()

    def calculate_fitness(self):
        return (self.gene[0] - 1) ** 2 +
sum(
            i * (2 * self.gene[i] ** 2 -
self.gene[i - 1]) ** 2 for i in range(1,
N)
        )

# Initialize population
def initialize_population():
    return [Individual() for _ in
range(POP_SIZE)]

# Selection function (simple elitism-
based)
def elitism_selection(population):
    # Select the best individuals to
form the next generation
    return sorted(population, key=lambda
ind: ind.fitness)[:POP_SIZE]

# Two-point crossover
def crossover(parent1, parent2):
    child1, child2 =
copy.deepcopy(parent1),
copy.deepcopy(parent2)
    if random.random() < CROSS_PROB:
        point1, point2 =
sorted(random.sample(range(N), 2))
        child1.gene[point1:point2],
child2.gene[point1:point2] =
parent2.gene[point1:point2],
parent1.gene[point1:point2]
    child1.fitness =
child1.calculate_fitness()
    child2.fitness =
child2.calculate_fitness()
    return child1, child2

# Mutation function
def mutate(individual, mut_rate,
mut_step):
    for i in range(N):
        if random.random() < mut_rate:
```

```
            alter = random.uniform(-
mut_step, mut_step)
            individual.gene[i] += alter
            # Clamp the gene to the
defined limits
            individual.gene[i] =
max(MIN_VAL, min(MAX_VAL,
individual.gene[i]))
    individual.fitness =
individual.calculate_fitness()
    return individual

# Genetic Algorithm
def run_ga(gens, mut_rate, mut_step):
    population = initialize_population()
    best_individual = min(population,
key=lambda ind: ind.fitness)

    # Lists for storing fitness values
to plot later
    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        # Select parents using elitism
        parents =
elitism_selection(population)

        # Crossover and mutation to
produce offspring
        offspring = []
        for i in range(0, POP_SIZE, 2):
            parent1, parent2 =
parents[i], parents[min(i+1, POP_SIZE-
1)]
            child1, child2 =
crossover(parent1, parent2)

offspring.extend([mutate(child1,
mut_rate, mut_step), mutate(child2,
mut_rate, mut_step)])

        # Elitism: Preserve the best
individual from the previous generation
        offspring[random.randint(0,
POP_SIZE - 1)] =
copy.deepcopy(best_individual)

        # Update population and track
the best individual
        population = offspring
        current_best = min(population,
key=lambda ind: ind.fitness)
        if current_best.fitness <
best_individual.fitness:
            best_individual =
copy.deepcopy(current_best)

        # Record statistics for this
generation

best_fitness_over_time.append(best_indiv
idual.fitness)

mean_fitness_over_time.append(mean([ind.
fitness for ind in population]))

    return best_individual.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Grid Search for Hyperparameter Tuning
def grid_search():
    results = []
    best_overall_fitness = float('inf')
    best_params = None
    gens_list = []
    mut_rate_list = []
    mut_step_list = []
    fitness_list = []

    for gens in GENS_RANGE:
        for mut_rate in MUT_RATES:
            for mut_step in MUT_STEPS:
                print(f"Testing
GENS={gens}, MUT_RATE={mut_rate},
MUT_STEP={mut_step}")
```

```
                best_fitness,
best_fitness_over_time,
mean_fitness_over_time = run_ga(gens,
mut_rate, mut_step)
                print(f"Best Fitness:
{best_fitness}")

                # Record results for
this configuration
                results.append((gens,
mut_rate, mut_step, best_fitness))
                gens_list.append(gens)

mut_rate_list.append(mut_rate)

mut_step_list.append(mut_step)

fitness_list.append(best_fitness)

                # Update best parameters
if this configuration is the best so far
                if best_fitness <
best_overall_fitness:
                    best_overall_fitness
= best_fitness
                    best_params = (gens,
mut_rate, mut_step)
                    best_fitness_plot =
best_fitness_over_time
                    mean_fitness_plot =
mean_fitness_over_time

    # Display the best parameters and
corresponding fitness
    print("\nBest Hyperparameters
Found:")
    print(f"GENS={best_params[0]},
MUT_RATE={best_params[1]},
MUT_STEP={best_params[2]}")
    print(f"Best Fitness:
{best_overall_fitness}")
    # Process time ends
    time_end = time.time()
```

```
    print("Process time:", time_end -
time_start)

    # 3D Scatter Plot
    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111,
projection='3d')
    sc = ax.scatter(gens_list,
mut_rate_list, mut_step_list,
c=fitness_list, cmap='viridis', s=60)
    ax.set_xlabel('Number of Generations
(GENS)')
    ax.set_ylabel('Mutation Rate
(MUT_RATE)')
    ax.set_zlabel('Mutation Step
(MUT_STEP)')
    ax.set_title('3D Scatter Plot of
Best Fitness Values')
    plt.colorbar(sc, label='Fitness
Value')
    plt.show()


    # Plot best fitness over generations
    plt.figure(figsize=(10, 6))
    plt.plot(best_fitness_plot,
label='Best Fitness')
    plt.plot(mean_fitness_plot,
label='Mean Fitness')
    plt.xlabel('Generation')
    plt.ylabel('Fitness')
    plt.legend()
    plt.title(f'Best Fitness Over
Generations (GENS={best_params[0]},
MUT_RATE={best_params[1]},
MUT_STEP={best_params[2]})')
    plt.show()

# Run the Grid Search
grid_search()
```

Comparison code for function 1 (see below):

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
```

```python
import matplotlib.pyplot as plt
import random
from statistics import mean
import copy
import time

# Function parameters
N = 20                # Number of genes
per individual (dimension d = 20)
POP_SIZE = 60         # Population size
MIN_VAL = -10         # Minimum gene
value for Function 1
MAX_VAL = 10          # Maximum gene
value for Function 1
CROSS_PROB = 0.9      # Crossover
probability

class Individual:
    def __init__(self):
        self.gene =
[random.uniform(MIN_VAL, MAX_VAL) for _
in range(N)]
        self.fitness =
self.calculate_fitness()

    def calculate_fitness(self):
        return (self.gene[0] - 1) ** 2 +
sum(
            i * (2 * self.gene[i] ** 2 -
self.gene[i - 1]) ** 2 for i in range(1,
N)
        )

# Helper functions for GA
def crossover(parent1, parent2):
    child1, child2 =
copy.deepcopy(parent1),
copy.deepcopy(parent2)
    if random.random() < CROSS_PROB:
        point1, point2 =
sorted(random.sample(range(N), 2))
        child1.gene[point1:point2],
child2.gene[point1:point2] =
parent2.gene[point1:point2],
parent1.gene[point1:point2]
    child1.fitness =
child1.calculate_fitness()
    child2.fitness =
child2.calculate_fitness()
    return child1, child2

def mutate(individual, mut_rate,
mut_step):
    for i in range(N):
        if random.random() < mut_rate:
            alter = random.uniform(-
mut_step, mut_step)
            individual.gene[i] += alter
            individual.gene[i] =
max(MIN_VAL, min(MAX_VAL,
individual.gene[i]))
    individual.fitness =
individual.calculate_fitness()
    return individual

# Genetic Algorithm
def run_ga(gens, mut_rate, mut_step):
    population = [Individual() for _ in
range(POP_SIZE)]
    best_individual = min(population,
key=lambda ind: ind.fitness)

    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        parents = sorted(population,
key=lambda ind: ind.fitness)[:POP_SIZE]
        offspring = []
        for i in range(0, POP_SIZE, 2):
            parent1, parent2 =
parents[i], parents[min(i+1, POP_SIZE-
1)]
            child1, child2 =
crossover(parent1, parent2)

offspring.extend([mutate(child1,
```

```
mut_rate, mut_step), mutate(child2,
mut_rate, mut_step)])
        offspring[random.randint(0,
POP_SIZE - 1)] =
copy.deepcopy(best_individual)
        population = offspring
        current_best = min(population,
key=lambda ind: ind.fitness)
        if current_best.fitness <
best_individual.fitness:
            best_individual =
copy.deepcopy(current_best)

best_fitness_over_time.append(best_indiv
idual.fitness)

mean_fitness_over_time.append(mean([ind.
fitness for ind in population]))

    return best_individual.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Simulated Annealing
def run_sa(max_iter, initial_temp,
cooling_rate):
    current_solution = Individual()
    best_solution =
copy.deepcopy(current_solution)
    temp = initial_temp

    best_fitness_over_time = []

    for i in range(max_iter):
        neighbor =
copy.deepcopy(current_solution)
        idx = random.randint(0, N - 1)
        neighbor.gene[idx] +=
random.uniform(-1, 1)
        neighbor.gene[idx] =
max(MIN_VAL, min(MAX_VAL,
neighbor.gene[idx]))
        neighbor.fitness =
neighbor.calculate_fitness()
```

```
        delta_fitness = neighbor.fitness
- current_solution.fitness
        if delta_fitness < 0 or
random.random() < np.exp(-delta_fitness
/ temp):
            current_solution = neighbor
            if current_solution.fitness
< best_solution.fitness:
                best_solution =
copy.deepcopy(current_solution)


best_fitness_over_time.append(best_solut
ion.fitness)
        temp *= cooling_rate

    return best_solution.fitness,
best_fitness_over_time

# Random Hill Climbing
def run_rhc(max_iter):
    current_solution = Individual()
    best_solution =
copy.deepcopy(current_solution)

    best_fitness_over_time = []

    for i in range(max_iter):
        neighbor =
copy.deepcopy(current_solution)
        idx = random.randint(0, N - 1)
        neighbor.gene[idx] +=
random.uniform(-1, 1)
        neighbor.gene[idx] =
max(MIN_VAL, min(MAX_VAL,
neighbor.gene[idx]))
        neighbor.fitness =
neighbor.calculate_fitness()

        if neighbor.fitness <
current_solution.fitness:
            current_solution = neighbor
```

```
            if current_solution.fitness
< best_solution.fitness:
                best_solution =
copy.deepcopy(current_solution)


best_fitness_over_time.append(best_solut
ion.fitness)

    return best_solution.fitness,
best_fitness_over_time

# Comparison of GA, SA, and RHC
def compare_algorithms():
    print("Running Genetic
Algorithm...")
    best_ga_fitness, ga_fitness_plot, _
= run_ga(gens=500, mut_rate=0.6,
mut_step=0.2)
    print(f"Best GA Fitness:
{best_ga_fitness}")

    print("\nRunning Simulated
Annealing...")
    best_sa_fitness, sa_fitness_plot =
run_sa(max_iter=500, initial_temp=100,
cooling_rate=0.95)
    print(f"Best SA Fitness:
{best_sa_fitness}")

    print("\nRunning Random Hill
Climbing...")
    best_rhc_fitness, rhc_fitness_plot =
run_rhc(max_iter=500)
    print(f"Best RHC Fitness:
{best_rhc_fitness}")

    # Plot comparison
    plt.figure(figsize=(10, 6))
    plt.plot(ga_fitness_plot, label='GA
Best Fitness')
    plt.plot(sa_fitness_plot, label='SA
Best Fitness', linestyle='dashed')
    plt.plot(rhc_fitness_plot,
label='RHC Best Fitness',
linestyle='dotted')
    plt.xlabel('Iteration/Generation')
    plt.ylabel('Fitness')
    plt.legend()
    plt.title('Comparison of GA, SA, and
RHC Performance')
    plt.show()

# Run the comparison
compare_algorithms()
```

Comparison code for function 2(see below):

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
import random
import copy
import time
from statistics import mean

# Constants specific to Function 2
N = 20                # Number of genes
per individual (dimension d = 20)
POP_SIZE = 100         # Population size
MIN_VAL = -500        # Minimum gene
value
MAX_VAL = 500         # Maximum gene
value
YOU = 61              # The last two
digits of the student number

# Common parameters for all algorithms
GENS = 500            # Number of
generations
MUT_RATE = 0.4        # Mutation rate for
GA and RHC
MUT_STEP = 0.7        # Mutation step for
GA and RHC
INITIAL_TEMP = 1000   # Initial
temperature for SA
```

```python
COOLING_RATE = 0.99  # Cooling rate for
SA
CROSS_PROB = 0.9     # Crossover
probability for GA

# Process time starts
time_start = time.time()

class Individual:
    def __init__(self):
        self.gene =
[random.uniform(MIN_VAL, MAX_VAL) for _
in range(N)]
        self.fitness =
self.calculate_fitness()

    def calculate_fitness(self):
        # f(x) = 418.9829 * d - sum(x_i
* sin(sqrt(|x_i|))) for i=1 to d + YOU
        sum_term = sum(self.gene[i] *
np.sin(np.sqrt(abs(self.gene[i]))) for i
in range(N))
        return ((418.9829 * N) -
sum_term ) + YOU

# Initialize population for GA
def initialize_population():
    return [Individual() for _ in
range(POP_SIZE)]

# Selection function (simple elitism-
based)
def elitism_selection(population):
    return sorted(population, key=lambda
ind: ind.fitness)[:POP_SIZE]

# Two-point crossover for GA
def crossover(parent1, parent2):
    child1, child2 =
copy.deepcopy(parent1),
copy.deepcopy(parent2)
    if random.random() < CROSS_PROB:
        point1, point2 =
sorted(random.sample(range(N), 2))
        child1.gene[point1:point2],
child2.gene[point1:point2] =
parent2.gene[point1:point2],
parent1.gene[point1:point2]
    child1.fitness =
child1.calculate_fitness()
    child2.fitness =
child2.calculate_fitness()
    return child1, child2

# Mutation function for GA and RHC
def mutate(individual, mut_rate,
mut_step):
    for i in range(N):
        if random.random() < mut_rate:
            alter = random.uniform(-
mut_step, mut_step)
            individual.gene[i] += alter
            individual.gene[i] =
max(MIN_VAL, min(MAX_VAL,
individual.gene[i]))
    individual.fitness =
individual.calculate_fitness()
    return individual

# Genetic Algorithm (GA)
def run_ga(gens, mut_rate, mut_step):
    population = initialize_population()
    best_individual = min(population,
key=lambda ind: ind.fitness)

    # Lists for tracking fitness over
time
    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        # Select parents using elitism
        parents =
elitism_selection(population)

        # Crossover and mutation to
produce offspring
        offspring = []
```

```
        for i in range(0, POP_SIZE, 2):
            parent1, parent2 =
parents[i], parents[min(i+1, POP_SIZE-
1)]
            child1, child2 =
crossover(parent1, parent2)

offspring.extend([mutate(child1,
mut_rate, mut_step), mutate(child2,
mut_rate, mut_step)])

        # Elitism: Preserve the best
individual from the previous generation
        offspring[random.randint(0,
POP_SIZE - 1)] =
copy.deepcopy(best_individual)

        # Update population and track
the best individual
        population = offspring
        current_best = min(population,
key=lambda ind: ind.fitness)
        if current_best.fitness <
best_individual.fitness:
            best_individual =
copy.deepcopy(current_best)

        # Record statistics for this
generation

best_fitness_over_time.append(best_indiv
idual.fitness)

mean_fitness_over_time.append(mean([ind.
fitness for ind in population]))

    return best_individual.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Simulated Annealing (SA)
def run_sa(gens, initial_temp,
cooling_rate):
    current_solution = Individual()
    best_solution = current_solution

    # Lists for tracking fitness over
time
    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        temp = initial_temp *
(cooling_rate ** gen)
        new_solution = Individual()

        # Calculate fitness difference
        delta_fitness =
current_solution.fitness -
new_solution.fitness

        # Acceptance probability
        if delta_fitness > 0 or
random.random() < np.exp(delta_fitness /
temp):
            current_solution =
new_solution
            if current_solution.fitness
< best_solution.fitness:
                best_solution =
current_solution

        # Record statistics

best_fitness_over_time.append(best_solut
ion.fitness)

mean_fitness_over_time.append(current_so
lution.fitness)

    return best_solution.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Random Hill Climbing (RHC)
def run_rhc(gens, mut_rate, mut_step):
    current_solution = Individual()
    best_solution = current_solution
```

```python
    # Lists for tracking fitness over
time
    best_fitness_over_time = []
    mean_fitness_over_time = []

    for gen in range(gens):
        new_solution =
copy.deepcopy(current_solution)

        # Random mutation
        for i in range(N):
            if random.random() <
mut_rate:  # Mutation with a set rate
                new_solution.gene[i] +=
random.uniform(-mut_step, mut_step)
                new_solution.gene[i] =
max(MIN_VAL, min(MAX_VAL,
new_solution.gene[i]))

        new_solution.fitness =
new_solution.calculate_fitness()

        # If the new solution is better,
replace the current solution
        if new_solution.fitness <
current_solution.fitness:
            current_solution =
new_solution
            if current_solution.fitness
< best_solution.fitness:
                best_solution =
current_solution

        # Record statistics

best_fitness_over_time.append(best_solut
ion.fitness)

mean_fitness_over_time.append(current_so
lution.fitness)
```

```python
    return best_solution.fitness,
best_fitness_over_time,
mean_fitness_over_time

# Run all algorithms and compare results
def compare_algorithms():
    print("Running Genetic Algorithm
(GA)...")
    ga_fitness,
ga_best_fitness_over_time,
ga_mean_fitness_over_time = run_ga(GENS,
MUT_RATE, MUT_STEP)
    print(f"GA Best Fitness:
{ga_fitness}")

    print("Running Simulated Annealing
(SA)...")
    sa_fitness,
sa_best_fitness_over_time,
sa_mean_fitness_over_time = run_sa(GENS,
INITIAL_TEMP, COOLING_RATE)
    print(f"SA Best Fitness:
{sa_fitness}")

    print("Running Random Hill Climbing
(RHC)...")
    rhc_fitness,
rhc_best_fitness_over_time,
rhc_mean_fitness_over_time =
run_rhc(GENS, MUT_RATE, MUT_STEP)
    print(f"RHC Best Fitness:
{rhc_fitness}")

    # Plotting fitness over generations
for all algorithms
    plt.figure(figsize=(12, 6))
    plt.plot(ga_best_fitness_over_time,
label='GA Best Fitness')
    plt.plot(sa_best_fitness_over_time,
label='SA Best Fitness')
    plt.plot(rhc_best_fitness_over_time,
label='RHC Best Fitness')
    plt.xlabel('Generation')
    plt.ylabel('Fitness')
```

```
    plt.title('Comparison of Algorithms
Over Generations')
    plt.legend()
    plt.show()

# Run the comparison
compare_algorithms()

# Process time ends
time_end = time.time()
print("Process time:", time_end -
time_start)
```